

Pattern Binding & Regex Operators

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>\$str =~ /pattern/</code>	Pattern binding match operator — tests if \$str matches regular expression (returns true/false in scalar context) EXAMPLE: <code>if (\$email =~ /^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}\$/i) { ... }</code>
<code>\$str !~ /pattern/</code>	Negative pattern binding match operator — returns true if \$str does NOT match regular expression EXAMPLE: <code>if (\$password !~ /\d/) { die "Password needs a digit"; }</code>
<code>\$str =~ s/pattern/ replacement/flags</code>	Substitution operator — replaces matched pattern with replacement string (flags: /g, /i, /e, /r) EXAMPLE: <code>\$text =~ s/\bapple\b/orange/gi;</code>
<code>\$str =~ tr/SEARCH/REPLACE/ flags</code>	Transliteration operator (synonym: y///) — translates all characters in search list to replace list (/d delete, /s squash, /c complement) EXAMPLE: <code>\$dna =~ tr/atcg/tagc/; # Transliterate DNA bases</code>
<code>qr/pattern/flags</code>	Quote regex operator — compiles pattern and flags into a reusable, pre-compiled Regexp object EXAMPLE: <code>my \$rx = qr/\b[A-Z]{3}-\d{4}\b/i;</code>
<code>split(/pattern/, \$str [, limit])</code>	Splits \$str into a list of substrings separated by delimiter matches EXAMPLE: <code>my @fields = split(/[,;]\s*/, \$csv_line);</code>
<code>my @matches = (\$str =~ / pattern/g)</code>	List context match with /g flag — returns all captured groups or full matches as a list EXAMPLE: <code>my @words = (\$text =~ /\b\w+\b/g);</code>
<code>while (\$str =~ /pattern/g) { ... }</code>	Scalar context match in while loop — iterates match by match through string, tracking offset via pos(\$str) EXAMPLE: <code>while (\$log =~ /\(\d{4}-\d{2}-\d{2}\)/g) { say "Found date at " . pos(\$log); }</code>

Special Regex Variables & Pointers

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>\$1, \$2, \$3 ...</code>	Contains the text captured by the 1st, 2nd, 3rd parenthesized capturing group from last successful match EXAMPLE: <code>if (\$str =~ /(\w+):(\d+)/) { my (\$k, \$v) = (\$1, \$2); }</code>
<code>\$& (or \${^MATCH})</code>	The entire string matched by the last successful pattern (use /p flag or \${^MATCH} to avoid performance penalty) EXAMPLE: <code>\$str =~ /\d+/; say "Matched: \$&;"</code>
<code>\$` and \$(' (or \${^PREMATCH}, \${^POSTMATCH})</code>	The string preceding (\$`) and following \$(') the last successful match  EXAMPLE: <code>\$str =~ /:/; say "Before: \$`, After: \$('";</code>
<code>+\$ (or \${^LAST_PAREN_MATCH})</code>	Returns the text matched by the highest-numbered (last) capturing group EXAMPLE: <code>\$str =~ /(foo) (bar) (baz)/; say "Last: \$+";</code>
<code>%+ and %-</code>	Hashes holding named capture groups: <code>#{name}</code> contains captured text; <code>#{name}[]</code> contains arrayref of all captures for that name EXAMPLE: <code>if (\$str =~ /(?!<year>\d{4})-(?!<month>\d{2})/) { say \$+{year}; }</code>
<code>@- and @+</code>	Arrays holding start (\$-) and end (\$) byte/character offsets in target string for \$& (index 0) and capture groups (indices 1..N) EXAMPLE: <code>my \$start = \$-[0]; my \$end = \$+[0]; # Offsets of full match</code>
<code>pos(\$str)</code>	Returns or sets the current offset position where the next /g search will resume EXAMPLE: <code>pos(\$str) = 0; # Reset regex cursor</code>
<code>\K</code>	Keep-out escape sequence: resets match start; drops everything matched to the left from \$& and s/// replacement EXAMPLE: <code>\$str =~ s/foo\Kbar/baz/; # replaces only "bar" when preceded by "foo"</code>

Pattern Modifiers & Flags

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>/g</code>	Global — match or replace all occurrences throughout the target string EXAMPLE: <code>\$str =~ s/\s+/ /g;</code>
<code>/i</code>	Case-insensitive matching (ignores uppercase vs lowercase distinction) EXAMPLE: <code>\$str =~ /hello/i</code> → matches "HELLO", "Hello", "hello"
<code>/m</code>	Multiline mode — <code>^</code> and <code>\$</code> match beginning and end of each physical line within string EXAMPLE: <code>\$str =~ /^#\s*/m;</code>
<code>/s</code>	Single-line / Dot-all mode — permits the dot (.) metacharacter to match newline (<code>\n</code>) EXAMPLE: <code>\$str =~ /<div.*?>(.*?)<\div>/s;</code>
<code>/x</code> and <code>/xx</code>	Extended / Verbose mode — ignores unescaped whitespace and permits # comments; <code>/xx</code> also ignores whitespace inside bracket classes EXAMPLE: <code>\$str =~ / \d{4} # Year \n -\d{2} # Month /x;</code>
<code>/e</code>	Evaluate replacement string as executable Perl code expression (can be chained e.g. <code>/ee</code>) EXAMPLE: <code>\$str =~ s/(\d+)/\$1 * 2/ge; # Doubles all integers</code>
<code>/r</code>	Non-destructive substitution (Perl 5.14+) — returns modified copy of string without altering original variable EXAMPLE: <code>my \$slug = \$title =~ s/[^a-z0-9]+/-/gir;</code>
<code>/a</code> and <code>/aa</code>	ASCII-only matching rules for <code>\d</code> , <code>\w</code> , <code>\s</code> , and POSIX classes; <code>/aa</code> enforces ASCII casing EXAMPLE: <code>\$str =~ /\w+/a;</code>
<code>/u</code>	Unicode rules — enables full Unicode character semantics on character classes EXAMPLE: <code>\$str =~ /\w+/u;</code>
<code>/o</code>	Compile pattern only once when compiling the script (use with caution if variable interpolation changes) EXAMPLE: <code>\$str =~ /\$pattern/o;</code>

Advanced Constructs & PCRE Features

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>(?<name>pattern) /</code> <code>(?'name'pattern)</code>	Named capture group — captures matched substring and stores it in <code>\$+{name}</code> EXAMPLE: <code>(?<user>\w+)@(?<host>[\w.-]+)</code>
<code>\g{1}, \g{-1}, \g{name},</code> <code>\k<name></code>	Backreferences by absolute number (<code>\g{1}</code>), relative position (<code>\g{-1}</code> for previous group), or name (<code>\k<name></code>) EXAMPLE: <code>(['"])(.*?)\g{-2}</code>
<code>(?>pattern)</code> and <code>*+, ++, ?+</code>	Atomic grouping and possessive quantifiers — matches greedily and prevents backtracking into the group EXAMPLE: <code>(?>\w+):</code>
<code>(? pattern1 pattern2)</code>	Branch reset group — resets capturing group numbering across alternatives so each branch uses the same <code>\$1, \$2</code> EXAMPLE: <code>(? (\d{4})-(\d{2}) ([a-z]+)/(\d+))</code>
<code>(?(condition)yes-pattern no-pattern)</code>	Conditional match — if group number/name or lookahead condition evaluates true, matches yes-pattern, else no-pattern EXAMPLE: <code>^(?<paren>\(\)?\d{3})(?(paren)\) -)(\d{4})\$</code>
<code>(?R)</code> or <code>(?0) / (?1) / (?&name)</code>	Recursive regex and subroutine calls — <code>(?R)</code> recurses the entire pattern; <code>(?1)</code> recurses group 1; <code>(?&name)</code> recurses named group EXAMPLE: <code>\((?:[^(] (?R))*\)</code>
<code>(*SKIP)(*FAIL) / (*SKIP)(*F)</code>	Backtracking control verbs — discards unwanted matches and forces regex engine to skip ahead EXAMPLE: <code>"[^"]*"(*SKIP)(*F) \bkeyword\b</code>
<code>(?{ code })</code> and <code>(??{ code })</code>	Embedded code blocks — <code>(?{ code })</code> executes arbitrary Perl code at match point; <code>(??{ code })</code> evaluates code and treats result as sub-pattern EXAMPLE: <code>(\w+)(?{ say "Saw \$^N" })</code>

Perl Character Classes & Unicode Properties

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>\p{Letter} / \p{L} / \p{Lu} / \p{Ll}</code>	Unicode General Category: all letters (<code>\p{L}</code>), uppercase letter (<code>\p{Lu}</code>), lowercase letter (<code>\p{Ll}</code>) EXAMPLE: <code>\p{Lu}\p{Ll}+</code>
<code>\p{Number} / \p{N} / \p{Nd}</code>	Unicode numbers: all numbers (<code>\p{N}</code>) or decimal digits (<code>\p{Nd}</code>) EXAMPLE: <code>\p{N}+</code>
<code>\p{Punctuation} / \p{P}</code>	Unicode punctuation characters: open (<code>\p{Ps}</code>), close (<code>\p{Pe}</code>), connector (<code>\p{Pc}</code>), dash (<code>\p{Pd}</code>) EXAMPLE: <code>\p{P}</code>
<code>\P{Property}</code>	Negated Unicode property — matches any character that does NOT possess the property EXAMPLE: <code>\P{L}+</code>
<code>\p{Script=Greek} / \p{Script=Latin}</code>	Unicode Script matching (e.g. <code>\p{Script=Arabic}</code> , <code>\p{Script=Cyrillic}</code> , <code>\p{Script=Han}</code>) EXAMPLE: <code>\p{Script=Greek}+</code>
<code>\X</code>	Extended Unicode grapheme cluster — matches a complete user-perceived character including combining marks/accents EXAMPLE: <code>\X</code>